

Critical Thinking Module 6: Real-Time Process Scheduling

Alexander Ricciardi

Colorado State University Global

CSC507: Foundations of OSs

Professor: Dr. Joseph Issa

Jun 21, 2026

Critical Thinking Module 6: Real-Time Process Scheduling

Operating Systems (OSs) use process scheduling to decide which runnable work receives CPU time, when it receives it, and how it shares the processor with other work. This matters when a file-processing job grows from one million rows to ten million rows. The program itself still performs a simple operation, reading integer values from an input file and writing doubled values to an output file, but the OS must coordinate storage reads, output writes, CPU execution, worker processes, and scheduling policy. In other words, the assignment is not only about writing a faster script. It is also about observing how the system behaves when the same file-processing workload is divided into different numbers of scheduled tasks.

This paper evaluates ten methods for processing a 10,000,000-row file on an Ubuntu desktop machine. The first five methods match the required assignment options: one normal process, 10 split files scheduled as real-time workers, 2 split files scheduled as real-time workers, 5 split files scheduled as real-time workers, and 20 split files scheduled as real-time workers. The remaining methods test other practical efficiency ideas, including normal split processing, CPU-count split processing, larger buffering, Python threads, and an awk streaming pipeline. The main conclusion is that Method 5, using 20 split files, was the fastest measured method at 0.725 seconds. However, the terminal output also showed 'real_time_workers=0' for every real-time method. Therefore, the measured result should be interpreted carefully: the split-file process design improved throughput, but this run did not actually prove a benefit from Linux real-time scheduling priority.

Test Machine and Benchmark Method

The benchmark was run on a system with Ubuntu 24.04.4 LTS and uses a 13th Gen Intel Core i9-13900KS processor. Linux reported 32 logical CPUs, 24 cores, 2 threads per core, and

one socket. The same hardware inspection reported 125 GiB of RAM and two NVMe solid-state drives: a Samsung SSD 990 PRO 4TB and a WD_BLACK SN770 2TB. These specifications are important because the file-processing task is able to benefit from parallel worker processes and fast storage. A smaller machine with fewer CPU cores or a slower disk could produce a different ranking. The benchmark script created or reused a 10,000,000-line input file. Each line contained one integer. For every method, the script doubled each integer and wrote the result to an output file. It then verified correctness by checking that every output line equaled the matching input line multiplied by 2. This verification matters because a fast result is not useful if the output is incomplete or incorrect. The benchmark also attempted to use Linux ‘`SCHED_RR`’ real-time scheduling for the methods labeled real-time. Linux scheduling documentation explains that the scheduler chooses the next runnable thread based on scheduling policy and priority. Normal policies such as ‘`SCHED_OTHER`’ do not use a positive static priority, while real-time policies such as ‘`SCHED_FIFO`’ and ‘`SCHED_RR`’ use priorities in the range from 1 to 99 (Kerrisk, n.d.-a). The ‘`sched_setscheduler()`’ call can set the policy and priority for the calling thread, but it fails with ‘`EPERM`’ when the caller does not have the required privileges (Kerrisk, n.d.-b). That detail is important here because the benchmark output reported zero real-time-enabled workers for all real-time methods. The script requested real-time scheduling, but the actual run behaved like normal split-process scheduling.

Benchmark Files and Outputs

The benchmark was built from a small number of local files. The Python file contained the processing logic, the Bash file launched the run and captured system information, and the ‘`benchmark_output`’ folder stored the large input file, CSV evidence, system report, and final output files. This makes the result reproducible because the script, input file, result table, and

output files are all stored in the folder. Table 1 lists the main files used to create the outputs and the output files produced by the benchmark.

Table 1

Files used and produced by the benchmark

File	Lines or size	Description
cta6_realtime_scheduling_benchmark.py	34.7 KB	Main Python benchmark program. It creates or reuses the input file, runs the ten methods, verifies output correctness, and writes timing results.
run_cta6_benchmark.sh	3.5 KB	Bash launcher. It records Ubuntu system information and starts the Python benchmark with the selected line count and methods.
benchmark_output/cat6_input_10000000.txt	10,000,000 lines	Source data file containing one integer per line. Every method reads this same file.
benchmark_output/cat6_realtime_results.csv	11 lines	CSV file containing the measured time, split time, process time, combine time, scheduler status, and verification result for each method.
benchmark_output/cat6_system_info.txt	66 lines	Ubuntu hardware and OS report captured before the benchmark run. It includes CPU, memory, storage, and scheduler-limit information.
Screenshot Terminal CTA6.png	324 KB	Screenshot of the Ubuntu terminal after the benchmark completed. It visually confirms the command, method results, and final ranking.
method_outputs/m01_single_normal_output.txt	10,000,000 lines	Output from Method 1, the single normal-scheduler process.
method_outputs/m02_split10_realtime_output.txt	10,000,000 lines	Output from Method 2, the 10-part split with real-time scheduling requested.
method_outputs/m03_split2_realtime_output.txt	10,000,000 lines	Output from Method 3, the 2-part split with real-time scheduling requested.
method_outputs/m04_split5_realtime_output.txt	10,000,000 lines	Output from Method 4, the 5-part split with real-time scheduling requested.
method_outputs/m05_split20_realtime_output.txt	10,000,000 lines	Output from Method 5, the 20-part split with real-time scheduling requested. This was the fastest required method.
method_outputs/m06_split10_normal_output.txt	10,000,000 lines	Output from Method 6, the 10-part split using normal scheduling.
method_outputs/m07_split_auto_normal_output.txt	10,000,000 lines	Output from Method 7, the automatic CPU-count split using normal scheduling.
method_outputs/m08_single_large_buffer_output.txt	10,000,000 lines	Output from Method 8, the single-process run with a larger file buffer.

method_outputs/m09_split10_threads_output.txt	10,000,000 lines	Output from Method 9, the 10-part split using Python threads.
method_outputs/m10_awk_stream_output.txt	10,000,000 lines	Output from Method 10, the awk streaming pipeline.

Note: The table lists the script files and their output files used by the benchmark.

Figure 1

Ubuntu terminal output from the CAT-6 benchmark run

```
rowan@ai01:~/CSU/CSC-507/CTA-6$ ./run_cat6_benchmark.sh
System information written: /home/rowan/CSU/CSC-507/CTA-6/benchmark_output/cat6_system_info.txt
Running CAT-6 benchmark with LINE_COUNT=10000000 METHODS=all
Input file already exists: /home/rowan/CSU/CSC-507/CTA-6/benchmark_output/cat6_input_10000000.txt
Measured input line count: 10000000
Selected methods: m01_single_normal, m02_split10_realtime, m03_split2_realtime, m04_split5_realtime, m05_split20_realtime,
m06_split10_normal, m07_split_auto_normal, m08_single_large_buffer, m09_split10_threads, m10_awk_stream

Running 1. m01_single_normal
Run one normal-scheduler process on the full larger file.
Completed m01_single_normal: 2.065s, verified_lines=10000000, real_time_workers=0/1

Running 2. m02_split10_realtime
Split into 10 files, process each with real-time scheduling, then combine.
Completed m02_split10_realtime: 0.864s, verified_lines=10000000, real_time_workers=0/10

Running 3. m03_split2_realtime
Split into 2 files, process each with real-time scheduling, then combine.
Completed m03_split2_realtime: 1.640s, verified_lines=10000000, real_time_workers=0/2

Running 4. m04_split5_realtime
Split into 5 files, process each with real-time scheduling, then combine.
Completed m04_split5_realtime: 1.011s, verified_lines=10000000, real_time_workers=0/5

Running 5. m05_split20_realtime
Split into 20 files, process each with real-time scheduling, then combine.
Completed m05_split20_realtime: 0.725s, verified_lines=10000000, real_time_workers=0/20

Running 6. m06_split10_normal
Split into 10 files with normal scheduling to isolate the real-time effect.
Completed m06_split10_normal: 0.848s, verified_lines=10000000, real_time_workers=0/10

Running 7. m07_split_auto_normal
Split by available logical CPUs with normal scheduling.
Completed m07_split_auto_normal: 0.742s, verified_lines=10000000, real_time_workers=0/32

Running 8. m08_single_large_buffer
Run one normal process with a larger file buffer.
Completed m08_single_large_buffer: 2.064s, verified_lines=10000000, real_time_workers=0/1

Running 9. m09_split10_threads
Split into 10 files and process chunks with Python threads.
Completed m09_split10_threads: 2.683s, verified_lines=10000000, real_time_workers=0/10

Running 10. m10_awk_stream
Use the Unix awk stream processor as a compact optimized pipeline.
Completed m10_awk_stream: 0.769s, verified_lines=10000000, real_time_workers=0/1

Benchmark summary
-----
05 m05_split20_realtime: 0.725s (split=0.549s, process=0.161s, combine=0.012s, verified=True)
07 m07_split_auto_normal: 0.742s (split=0.560s, process=0.160s, combine=0.018s, verified=True)
10 m10_awk_stream: 0.769s (split=0.000s, process=0.769s, combine=0.000s, verified=True)
06 m06_split10_normal: 0.848s (split=0.553s, process=0.277s, combine=0.015s, verified=True)
02 m02_split10_realtime: 0.864s (split=0.561s, process=0.280s, combine=0.016s, verified=True)
04 m04_split5_realtime: 1.011s (split=0.553s, process=0.440s, combine=0.015s, verified=True)
03 m03_split2_realtime: 1.640s (split=0.543s, process=1.068s, combine=0.026s, verified=True)
08 m08_single_large_buffer: 2.064s (split=0.000s, process=2.064s, combine=0.000s, verified=True)
01 m01_single_normal: 2.065s (split=0.000s, process=2.064s, combine=0.000s, verified=True)
09 m09_split10_threads: 2.683s (split=0.562s, process=2.105s, combine=0.013s, verified=True)

CSV results written: /home/rowan/CSU/CSC-507/CTA-6/benchmark_output/cat6_realtime_results.csv
rowan@ai01:~/CSU/CSC-507/CTA-6$
```

Note. Figure 1 shows the Ubuntu terminal after running ‘./run_cat6_benchmark.sh’. All ten methods processed 10,000,000 lines and wrote the CSV results to ‘benchmark_output/cat6_realtime_results.csv’.

Benchmark Results

Table 2

File-processing benchmark results on the Ubuntu desktop machine

Rank	Method	Description	Total seconds	Split seconds	Process seconds	Combine seconds	Real-time workers	Verified
1	5	Split into 20 files and process workers marked as real-time	0.725	0.549	0.161	0.012	0/20	True
2	7	Split by available logical CPU count with normal scheduling	0.742	0.560	0.160	0.018	0/32	True
3	10	Use an awk streaming pipeline	0.769	0.000	0.769	0.000	0/1	True
4	6	Split into 10 files with normal scheduling	0.848	0.553	0.277	0.015	0/10	True
5	2	Split into 10 files and process workers marked as real-time	0.864	0.561	0.280	0.016	0/10	True
6	4	Split into 5 files and process workers marked as real-time	1.011	0.553	0.440	0.015	0/5	True
7	3	Split into 2 files and process workers marked as real-time	1.640	0.543	1.068	0.026	0/2	True
8	8	Single process with a larger buffer	2.064	0.000	2.064	0.000	0/1	True
9	1	Single normal process on the full larger file	2.065	0.000	2.064	0.000	0/1	True
10	9	Split into 10 files and process with Python threads	2.683	0.562	2.105	0.013	0/10	True

Note: All methods processed 10,000,000 input lines and verified successfully. The real-time worker count came from the benchmark terminal output. Because all real-time worker counts were zero, the real-time-labeled methods should be interpreted as split-process tests where real-time scheduling was requested but not granted.

The fastest required assignment method was Method 5, which split the file into 20 parts. It completed in 0.725 seconds. Method 2, the 10-file split, completed in 0.864 seconds. Method 4, the 5-file split, completed in 1.011 seconds. Method 3, the 2-file split, completed in 1.640 seconds. Method 1, the unsplit baseline, completed in 2.065 seconds. This shows a clear pattern: the larger file benefited from being divided into more independent work units, but only up to the point where splitting and merging overhead remained manageable. The split time stayed almost constant across the split methods, around 0.54 to 0.56 seconds. This means the cost of dividing the file was a fixed overhead paid before the parallel processing work could help. The processing phase changed more significantly. Two parts took 1.068 seconds of processing time, five parts took 0.440 seconds, ten parts took about 0.280 seconds, and 20 parts took 0.161 seconds. The CPU-count split method used 32 workers and had almost the same processing time as the 20-part method, but the total time was slightly slower because it had a little more split and combine overhead. In other words, the system had enough CPU capacity for many workers, but more workers were not automatically better.

Analysis of the Methods

Method 1 was the simplest approach. It used one normal process and finished in 2.065 seconds. This result is useful as the baseline because it shows how long the larger file takes when the program is run mostly as it already was. The single process did not pay split or combine overhead, but it also did not use the 32 logical CPUs very well. It mainly processed the file in one execution stream. Method 3 split the file into two pieces. It was faster than the baseline at 1.640 seconds, but it was still much slower than the 5-part, 10-part, and 20-part methods. The reason is that two workers provide only limited parallelism on a machine with 32 logical CPUs. The split overhead still existed, but the system did not receive enough independent workers to

use the available processor capacity well. Method 4 split the file into five pieces and finished in 1.011 seconds. This was a strong improvement over the two-part method because more workers could process separate chunks at the same time. However, it still did not use enough parallelism to match the 10-part or 20-part methods. Method 2 split the file into ten pieces and finished in 0.864 seconds. This was much faster than the baseline and the smaller split counts. However, Method 6, the same 10-part split without a real-time request, finished in 0.848 seconds. That small difference supports the conclusion that real-time priority was not the cause of the improvement in this run. The improvement came from splitting the work and using separate worker processes. Method 5 split the file into 20 pieces and finished in 0.725 seconds. This was the fastest required method and the fastest overall method. It appears to be the best balance for this workload and hardware. It created enough process-level parallelism to reduce the processing phase sharply, but not so many parts that split and combine overhead became too expensive. Hassan et al. (2025) describe multiprocessor task scheduling as a difficult optimization problem where response time, turnaround time, and missed deadlines are important measurements. Malabanan et al. (2025) also identify average turnaround time, average waiting time, and context switches as key CPU scheduling performance metrics. The benchmark is simpler than those research settings because the file chunks are independent and the only operation is doubling integers. Still, the same principle applies: performance depends on balancing available processors, worker count, scheduling overhead, and completion time.

Other Efficiency Improvements

In addition to the five previous options, other methods can be implemented to improve overall file-processing throughput. The first improvement that can be implemented is splitting the input file by available logical CPU so the workload could dynamically scale. As

demonstrated in the results, method 7 used the logical CPU count and completed in 0.742 seconds, which is similar to how Method 5 achieved its peak performance. This strategy is advantageous because it allows the script to adapt to the machine's hardware capabilities instead of using a fixed number of chunks. However, the benchmark also shows that using every logical CPU is not always the fastest approach. The 32-worker method was slightly slower than the 20-worker method, so a better production script would test several worker counts and select the fastest safe value. The second improvement is to use a compact Unix streaming tool when the operation is simple. Method 10 used `awk` and finished in 0.769 seconds without splitting or combining files. The GNU 'awk' manual explains that 'awk' supports normal arithmetic operations and operators within expressions (Free Software Foundation, n.d.). For this project's simple operation, 'awk' '`{print $1 * 2}`' works very well. This is useful because it keeps the overall workflow simple and does not add extra multiprocessing overhead. On the other hand, it is less flexible than Python code, which is more suited for processing more complex transformations, validation, and reporting. The third improvement is to use process-based parallelism instead of Python threads for this CPU-bound workload. Method 9, the Python-thread method, was the slowest method at 2.683 seconds. This is because the CPython's Global Interpreter Lock limits true parallel threading for CPU-bound Python work, and the Python documentation recommends using multiprocessing or 'ProcessPoolExecutor' (Python Software Foundation, 2026a). Additionally, the 'ProcessPoolExecutor' uses separate processes, it can also side-step the 'Global Interpreter Lock'; however, submitted objects must be pickable (Python Software Foundation, 2026b). In other words, threads are not automatically better just because there are more of them.

The fourth improvement can be defined as an approach that avoids repeated splitting when possible. In the benchmark, Method 5 spent 0.549 seconds splitting the input file, but only 0.161 seconds processing the chunks and 0.012 seconds combining them. Thus, if the upstream process can write data into 20 chunk files from the start, then a later processing step can avoid most of the split cost. This could reduce Method 5 splitting into 20 files processing from about 0.725 seconds to the combined processing and combination time of about 0.173 seconds. That is not a guaranteed or an exact prediction, but it is a reasonable expectation. The fifth improvement is to choose chunk count experimentally instead of assuming that more parallelism is always better. Round Robin scheduling has a similar scheduling tradeoff, that is, when the time quantum is too small, context-switch overhead increases, while a very large time quantum makes the behavior closer to FCFS (Barman et al, 2023). Moreover, the file-splitting process has a comparable structure. Too few chunks underuse the CPU. Too many chunks add scheduling, splitting, and combining overhead. A possible answer to these issues is not “always 20” or “always CPU count.” Thus, the best answer may be to benchmark a few safe choices and select the best one for the machine and workload. An inventive but still practical choice would be to add a small autotuning script. The script could test worker counts such as 2, 5, 10, 20, and 32 on a smaller sample file, and finally it runs the full 10,000,000-line job with the best worker count. This keeps the method simple enough for a student assignment, but it also follows a professional idea: measure the system before selecting the processing policy.

Conclusion

Method 5 is the fastest method, which splits the 10,000,000-row file into 20 parts and processes the chunks with separate workers. It completed in 0.725 seconds, compared with 2.065 seconds for the single-process baseline. The 10-part method was also strong at 0.864 seconds,

and the 5-part and 2-part methods showed intermediate improvement. This shows that chunking and process-level parallelism were the most effective on the Ubuntu desktop machine. However, the methods labeled real-time did request real-time scheduling; however, the benchmark output reported zero real-time-enabled workers. Therefore, this run does not prove that 'SCHED_RR' itself improved processing speed. It proves that splitting the file and using multiple worker processes improved speed. A separate run may be needed; a run that tests whether true real-time scheduling changes the result. Even then, real-time scheduling is usually about predictability and deadline control, not only throughput. Thus, the best added efficient methods/improvements are CPU-count-aware process splitting, simple Unix streaming tools such as 'awk' for simple transformations, avoiding Python threads for CPU-heavy parsing, pre-splitting input when possible, and autotuning the number of chunks. Ultimately, the best option is to use the 20-part split method for this specific machine and workload, while keeping the 'awk' method as a simple alternative and using CPU-count tuning when the script is moved to a different system.

References

- Barman, D., Paul, B., Bhattacharya, S., De, S., & Arya, G. P. (2023). Optimized round robin CPU scheduling for critical processes. *Global Journal of Computer Science and Technology: H Information & Technology*, 23(3), 12-18.
- Free Software Foundation. (n.d.). Arithmetic operators. *The GNU Awk User's Guide*.
https://www.gnu.org/software/gawk/manual/html_node/Arithmetic-Ops.html
- Hassan, H. E., Ibrahiem, K. H., & Madian, A. H. (2025). Optimizing multiprocessor performance in real-time systems using an innovative genetic algorithm approach. *Scientific Reports*, 15, Article 3842. <https://doi.org/10.1038/s41598-024-80910-4>
- Kerrisk, M. (n.d.-a). sched(7) - *Linux manual page*. man7.org. <https://man7.org/linux/man-pages/man7/sched.7.html>
- Kerrisk, M. (n.d.-b). sched_setscheduler(2) - *Linux manual page*. man7.org.
https://man7.org/linux/man-pages/man2/sched_setscheduler.2.html
- Malabanan, D. H. C., Foryasen, Z. L. D., Balay-odao, G. A., & Tandingan, D. R., Jr. (2025). Adaptive scheduling heuristic priority linear regression (ASH-PLR): A novel CPU scheduling algorithm using predictive priority levels. *Journal of Information Systems Engineering and Management*, 10(43s), 1160-1171.
- Python Software Foundation. (2026a). *threading - Thread-based parallelism*. Python 3.14.6 documentation. <https://docs.python.org/3/library/threading.html>

Python Software Foundation. (2026b). *concurrent.futures* - *Launching parallel tasks*. Python 3.14.6 documentation. <https://docs.python.org/3/library/concurrent.futures.html>